

Chapter Overview

- 10.1 Structured Programming
- 10.2 Types of Code Blocks
- 10.3 Sub Procedures
 - 10.3.1 Scope of Sub Procedure
 - 10.3.2 Passing Parameters to Sub Procedures
- 10.4 Scope of Variables
 - 10.4.1 Procedure-Level Variable
 - 10.4.2 Form-Level Variables
 - 10.4.3 Global Variables
- 10.5 Lifetime of Variables
 - 10.5.1 Lifetime of Global Variables
 - 10.5.2 Lifetime of Local Variables
- 10.6 Static Variables
- 10.7 Function Procedure
 - 10.7.1 Advantages of Functions
 - 10.7.2 Return Type
 - 10.7.3 Returning a Value from Function
- 10.8 Argument Passing Mechanism
 - 10.8.1 Pass by Reference
 - 10.8.2 Pass by Value
- 10.9 Optional Arguments
 - 10.9.1 Default Values for Optional Arguments
- 10.10 Named Arguments
- 10.11 Indefinite Arguments
- 10.12 Passing Array as Arguments
 - 10.12.1 Passing Elements of Array as Arguments
 - 10.12.2 Returning Array from Function
- 10.13 Passing User-Defined Data Type as Argument
- 10.14 Passing Objects as Argument
- 10.15 Exit Sub & Exit Function
- 10.16 Adding Procedures using Menu
- 10.17 Modules

Programming Exercise**Short Questions****Multiple Choices****Fill in the Blanks****True/False**

10.1 Structured Programming

Structured programming is a programming method in which long program is divided into small blocks of code. Each block has a unique name that can be used again and again. The use of small blocks of code makes programs easier to maintain and modify.

10.2 Types of Code Blocks

Visual Basic provides the facility to create following types of code blocks.

1. Event Procedure

Event procedure is a type of block that is attached to a particular event. It is executed whenever that event occurs.

2. Sub Procedure

Sub Procedure is a type of block that is executed by calling it with its name. It is also known as procedure.

3. Function Procedure

Function procedure is a type of block that is also executed by calling it with its name. It additionally can return some value after processing.

4. Module

A module is a collection of procedures and functions. It is just like a library of code where the programmer stores most frequently blocks of code. These modules can be used in many applications.

10.3 Sub Procedures

A sub procedure is a block of code that performs some operation. The code written in a sub procedure is executed only when it is called. A sub procedure can be called any number of times by just writing its name.

When a user-defined sub procedure is called, the control moves in that sub procedure. All statements of sub procedure are executed and then the control again moves back to the point where the sub procedure was called.

Syntax

The basic syntax for a procedure is as follows:

```
Private Sub ProcedureName(Parameters As Type, ...)  
    Statement(s)
```

```
End Sub
```

Here,

Private

It indicates the scope of sub procedure.

Sub

It is the keyword that is used to declare a sub procedure.

ProcedureName

It indicates the name of sub procedure to be declared.

Parameter:

Parameters are the values that are provided to sub procedure when it is called. Parameters are normally given in parentheses. If there are many parameters, these are separated by commas.

End sub

It indicates the end of sub procedure.

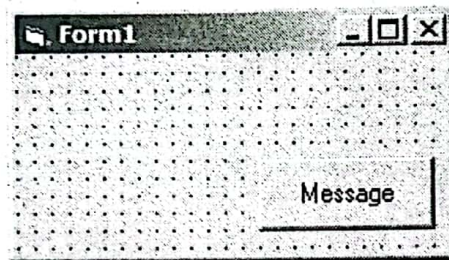
10.3.1 Scope of Sub Procedure

Normally, event procedures always start with **Private** keyword. Private means that the procedure is not accessible in other modules. On the other hand, a procedure with **Public** scope can be called from anywhere in the project. When creating procedures, **Public** is the default and **Public** keyword is not required.

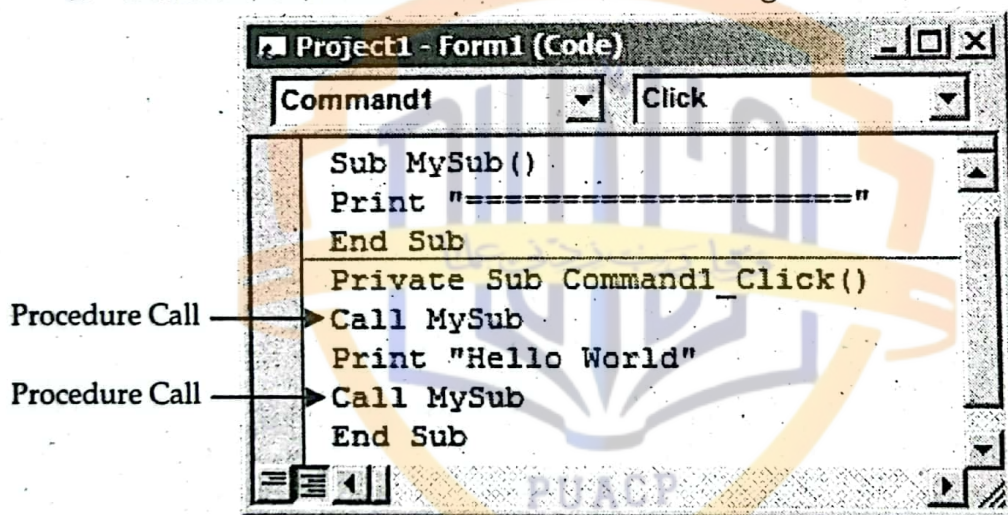
Program 10.1

Write a program that declares a subprogram to display a line. The program calls it before and after displaying a message "Hello World".

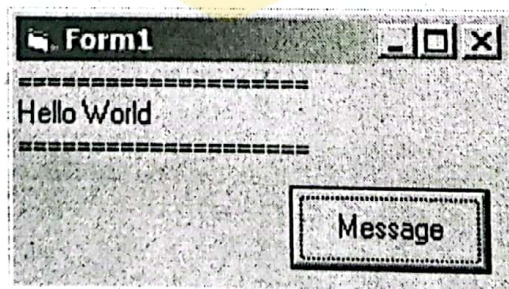
1. Start a new Standard EXE project.
2. Design the form as follows:



3. Double click the button and add the following code:



4. Run the project and click the button. The result will appear as follows:



How It Works?

The above example first declares a procedure **MySub**. The code written in procedure displays a line. The first line in the click event of the button calls this procedure that displays the line. After completing the procedure, the control automatically comes back on the second statement that prints a message. The third line again calls the procedure and the line is displayed again.

10.3.2 Passing Parameters to Sub Procedures

Parameters are the values that are passed to a sub procedure it is called. Parameters are normally given in parentheses. The number of parameters and their type is decided during procedure coding.

Program 10.2

Write a program that inputs two numbers in textboxes and passes the numbers to a sub procedure. The procedure adds the numbers and displays the result in a message box.

1. Start a new Standard EXE project.
2. Design the form as follows:

3. Double click the button and add the following code:

```

Public Sub Sum(x As Integer, y As Integer)
    Dim z As Integer
    z = x + y
    MsgBox "The sum is " & z
End Sub

Private Sub Command1_Click()
    Dim a As Integer, b As Integer
    a = Text1.Text
    b = Text2.Text
    Call Sum(a, b)
End Sub
  
```

4. Run the project.
5. Enter two numbers in textboxes and click Sum. The result will appear as follows:

How It Works?

The first line of **Sum** procedure indicates that the procedure will accept two integer parameters when called. The variables **x** and **y** in the parentheses are used to store the values of parameters. The event procedure calls **Sum** by providing two parameters **a** and **b**. The values of **a** and **b** are copied to **x** and **y** of **Sum** procedure respectively. The variable **z** in **Sum** is used to calculate the result. Finally, the result stored in **z** is displayed in message box.

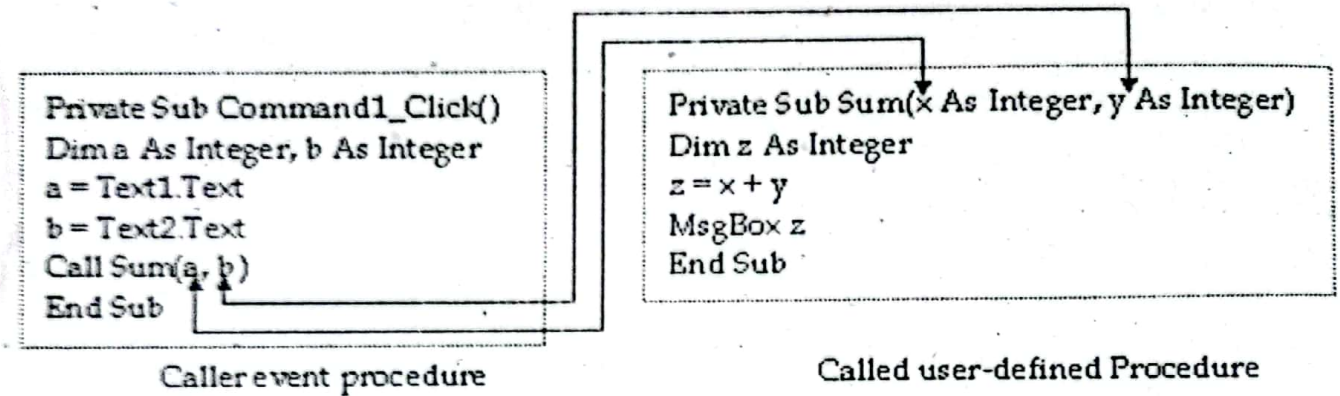


Figure: Parameter passing in procedures

10.4 Scope of Variables

The area from where a variable can be accessed is called its scope. Different types of variables with respect to their scope are as follows:

- Procedure-Level Variables
- Form-Level Variables
- Global Variables

10.4.1 Procedure-Level Variable

A variable that is declared inside a procedure can be used only in that particular procedure. It cannot be accessed or used from any other part of the application. This type of variable is also known as **local variable**.

10.4.2 Form-Level Variables

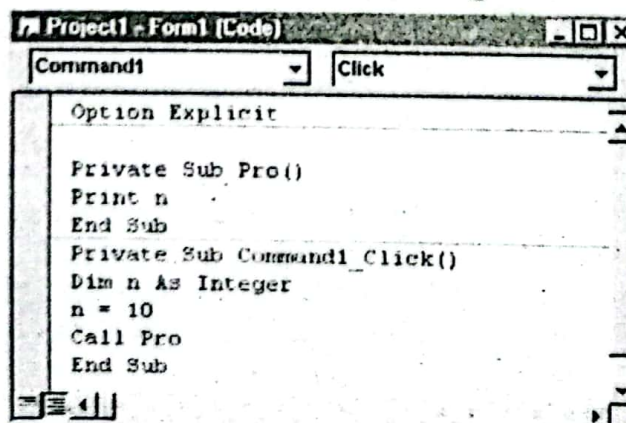
A variable that is declared in general declaration section of a form with **Dim** statement is called form-level variable. All procedures in that particular form can access this type of variables. But this variable cannot be used outside that form.

Program 10.3

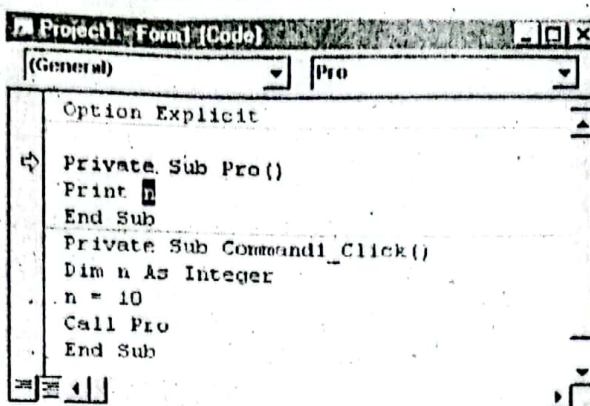
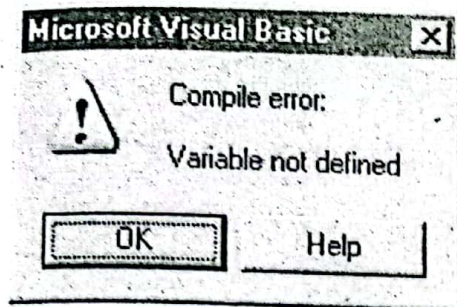
1. Start a new Standard EXE project.
2. Design the form as follows:



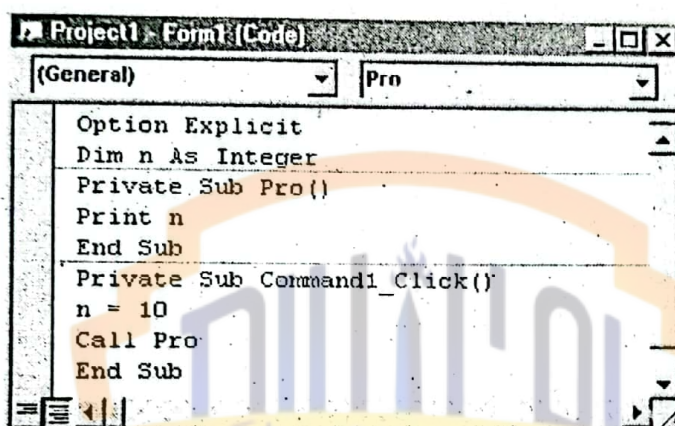
3. Double click the button and add the following code:



4. Run the project and click the button. An error will occur.
5. Click OK. The variable **n** will be selected to indicate the error.



6. Declare the variable **n** in general declaration section instead of event procedure as follows:



7. Run the project again and click the button. No error will occur and 10 will be displayed on the form as follows:



How It Works?

In the above example, **n** was declared in the event procedure. So its scope is limited to this event procedure. When the user-defined procedure **Pro** tried to use this variable, an error occurred. When **n** is declared in the general declaration section, it becomes form-level variable. So, all procedures can access it. Now **Pro** displays the value of **n** without any error.

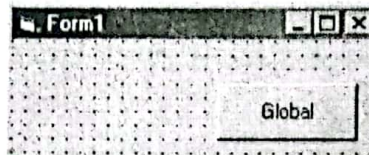
10.4.3 Global Variables

A variable declared in the general declaration section of a form or in module with **Public** statement is called global variable. This type of variable can be used by all procedures in the whole application.

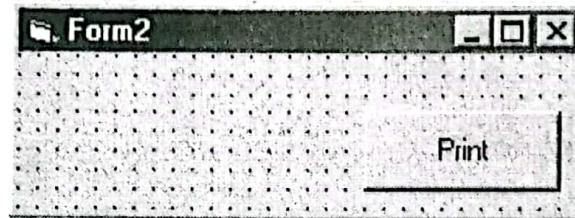
Program 10.4

Write a program that stores a value in a global variable in a form. The form contains a button that displays second form when it is clicked. The second form also contains a button. The value of global variable is shown when the user clicks this button.

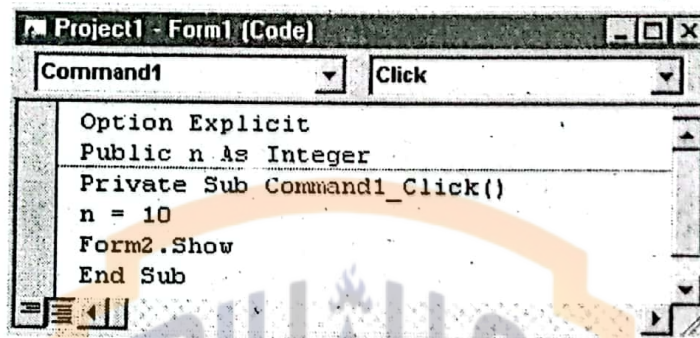
1. Start a new Standard EXE project.
2. Design the form as follows:



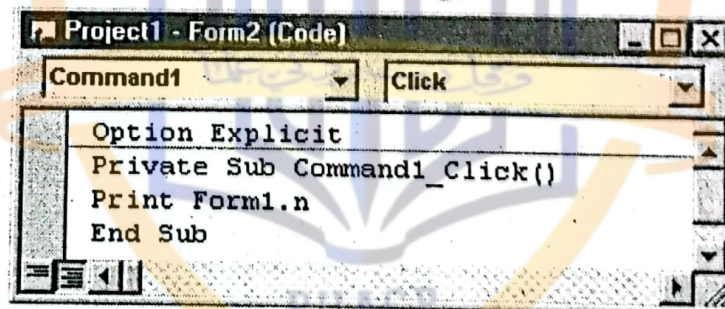
3. Add another form in the project and place a button on it as follows:



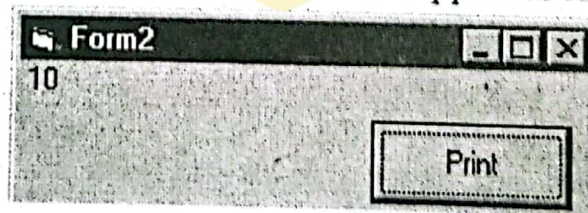
4. Double click the button on Form1 and add the following code:



5. Double on Form2 and add the following code:



6. Run the form and click the button on Form1. The Form2 will appear.
7. Click the button on Form2. The result will appear as follows:



How It Works?

The above example uses two forms. Form1 declares a variable `n` with value 10. As this variable is declared with `Public` statement, it can be accessed from any part of application. Form2 displays the value of `n`.

10.5 Lifetime of Variables

The time period for which a variable retains its value in the memory is known as its lifetime. Different types of variables have different lifetimes.

10.5.1 Lifetime of Global Variables

Global variables exist in the memory as long as the application is running. Any part of the application can access these variables. When the application ends, all global variables are destroyed.

10.5.2 Lifetime of Local Variables

Local variables or variables declared with Dim statement retain their values in the memory as long as the control remains in that particular procedure. As the control exits from the procedure, all variables declared in that procedure are also destroyed. If the procedure is called again, the local variables are again declared in the memory.

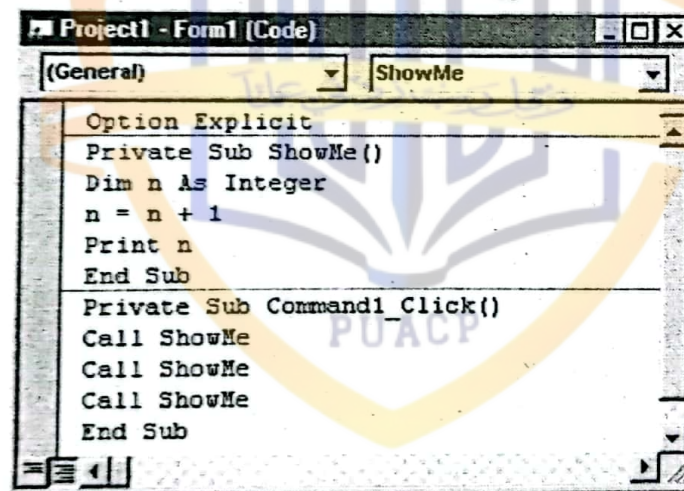
Program 10.5

Write a program that declares a subprogram. The subprogram declares a local variable, increments its value by 1 and displays it when it is called. The program calls it three times.

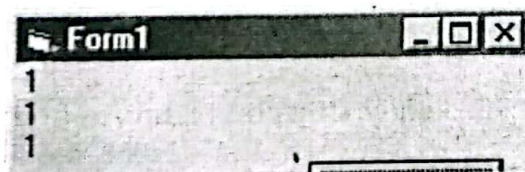
1. Start a new Standard EXE project.
2. Design the form as follows:



3. Double click the button and add the following code:



4. Run the project and click the button. The result will appear as follows:



How It Works?

The above example calls the procedure ShowMe three times. Each time 1 is displayed. The reason is that when the control enters the procedure, n is declared and initialized to 0. The statement "n = n + 1" makes it 1 and displays this value. When the control leaves the procedure, n is destroyed from the memory because it is a local variable. When the procedure is called again, the same process is repeated. That is why each time 1 is displayed.

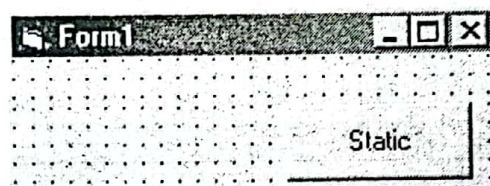
10.6 Static Variables

Static variable is a type of local variable that retains its value even if the control leaves the procedure. Such variable is declared with **Static** keyword. Sometimes, it is necessary to remember the value of last procedure call. It can be accomplished easily by using static variables.

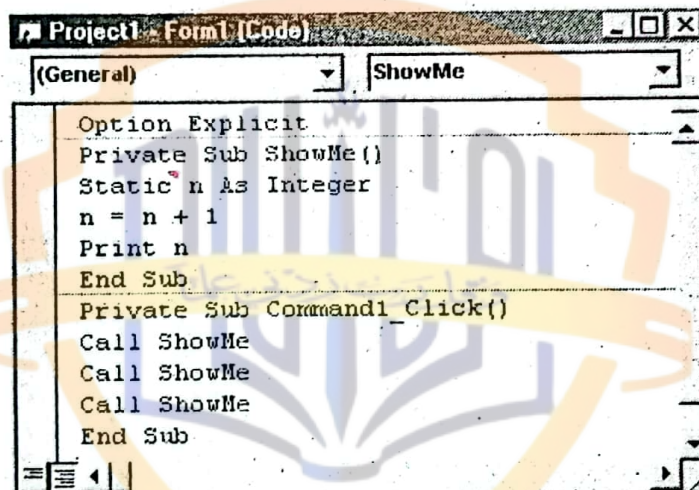
Program 10.6

Write a program that declares a subprogram. The subprogram declares static variable, increments its value by 1 and displays it when it is called. The program calls it three times.

1. Start a new Standard EXE project.
2. Design the form as follows:



3. Double click the button and add the following code:



4. Run the project and click the button. The result will appear as follows:



How It Works?

The above example uses a static variable **n**. The declaration statement is executed only in first procedure call. In first call, 1 is displayed. When the procedure is called for the second time, **n** already exists with value 1 so the declaration statement is skipped. **n** is incremented by 1 and 2 is displayed. In third call, **n** is incremented again by 1 and 3 is displayed.

10.7 Function Procedure

A **function procedure** is a block of code that performs some action. It is also called **function**. It is similar to the sub procedure. The difference is that it can **return** a value back to the position where it is called. The statements written in a function procedure are executed only if it is called by its name.

When a user-defined function procedure is called, the control moves in that function. All statements of the function are executed and then the control again moves back to the point where the function was called along with possible returned value.

10.7.1 Advantages of Functions

Some advantages of using functions in programs are as follows:

1. Easier to Code

A lengthy program can be divided into small functions. It is easier to write small functions instead of writing a long program. A function is written to solve a particular problem. A programmer can focus attention on a specific problem. It makes programming easier.

2. Easier to Modify

Each function has a unique name and is written as an independent block. If there is any error in the program, the change is made to particular function in which error exists. A small function is easier to modify than a large program.

3. Easier to Maintain

Functions are easier to maintain than long programs. Each function contains independent code. A change in the function does not affect other parts of program.

4. Reusability

The code written in functions can be reused as and when required. A function but can be executed many times. A function is written to solve a particular problem. Whenever that problem has to be solved, the function can be executed.

5. Less Programming Time

A program may consist of many functions. These functions are written as independent programs. Different programmers can work on different functions at the same time. It takes far less time to complete the program.

Syntax

The basic syntax for a function is as follows:

```
Private Function FunctionName(Parameter As Type, ...) As ReturnType
    Statement(s)
    FunctionName = ReturnValue
End Function
```

Here,

Private	It indicates the scope of function procedure.
Function	It is the keyword that is used to declare a function procedure.
FunctionName	It indicates the name of the function procedure to be declared.
Parameter:	Parameters are the values that are provided to function procedure when it is called. Parameters are normally given in parentheses. If there are many parameters, these are separated by commas.
Return-type:	It indicates the type of value that will be returned by function procedure. For example, Integer is used as return type if the function returns integer value.
End Function	It indicates the end of function procedure.

10.7.2 Return Type

The return type of the function indicates the type of value that will be returned by the function. The above figure shows that the function will return an integer value after processing. Although a function can return a value but it is not compulsory. If the function returns no value, the return type is not used in function definition. In this case, the function will work similar to a procedure.

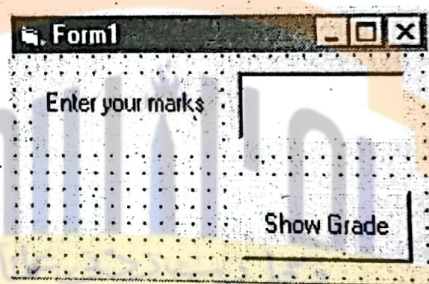
10.7.3 Returning a Value from Function

A function can return a value back to the calling point. A value is returned from function by simply assigning that value to the name of the function. Assume that the name of the function is a variable. Any value assigned to the name of the function is actually returned. In the above figure, 10 is returned to the point from where the function was called.

Program 10.7

Write a program that inputs marks in a textbox and passes the marks to function. The function calculates the grade and returns back. The grade is finally displayed in message box.

1. Start a new Standard EXE project.
2. Design the form as follows:



3. Double click the button and add the following code:

```

Project1 - Form1 (Code)
Command1 Click
Private Function grade(m As Integer) As String
    Dim g As String
    If m > 80 Then
        g = "A"
    ElseIf m > 60 Then
        g = "B"
    ElseIf m > 40 Then
        g = "C"
    Else
        g = "F"
    End If
    grade = g
End Function

Private Sub Command1_Click()
    Dim marks As Integer, gr As String
    marks = Text1.Text
    gr = grade(marks)
    MsgBox "Your grade is " & gr
End Sub
  
```

4. Run the project.

5. Enter marks in the text box and click the button. The grade will appear in the message box as follows:

How It Works?

The above example uses a user-defined function `grade` to find the grade of student. The function accepts marks as arguments, finds grade using IF statement and then returns the grade. The statement `"grade = g"` is used to return the grade value.

10.8 Argument Passing Mechanism

Arguments are passed to procedures and functions in two different ways:

- Pass by Reference
- Pass by Value

10.8.1 Pass by Reference

This is the default mechanism used by Visual Basic. When arguments are passed by reference to a procedure or function, the addresses of these arguments are provided to the called procedure. It means that the procedure accesses the actual value of the argument in the memory. If the procedure changes the value of the argument, the change is visible in the calling code.

Program 10.8

Write a program that inputs a number in a textbox and passes it to a function by reference. The function returns its square and sets its value to 0. The program displays the value of input before calling the function, its square and value after calling the function.

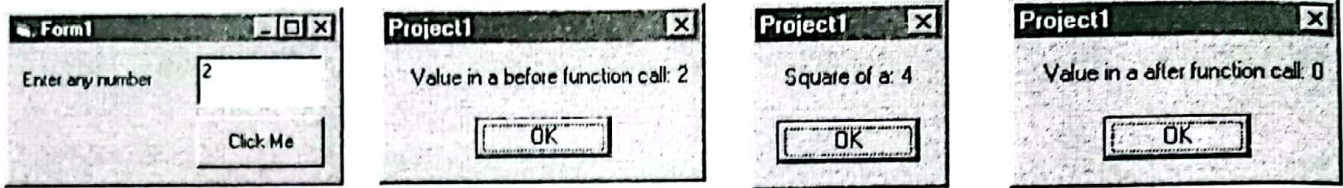
1. Start a new Standard EXE project.
2. Place a CommandButton on the form.
3. Add the following code in Code window:

```

Function Sqr(x As Integer) As Integer
    Sqr = x * x
    x = 0
End Function

Private Sub Command1_Click()
    Dim a As Integer, s As Integer
    a = Text1.Text
    MsgBox "Value in a before function call: " & a
    s = Sqr(a)
    MsgBox "Square of a: " & s
    MsgBox "Value in a after function call: " & a
End Sub
  
```


4. Run the project.
5. Enter any value in textbox and click the button. The result will appear as follows:



How It Works?

The above example gets a value from the user in the text box and passes it to the function by reference. The function returns the square of the number and assigns 0 to x. This assignment statement is actually storing 0 to n. The variable x in the function is actually another name of x. There is only one variable in the memory. In event procedure, its name is n and in user-defined function, its name is x.

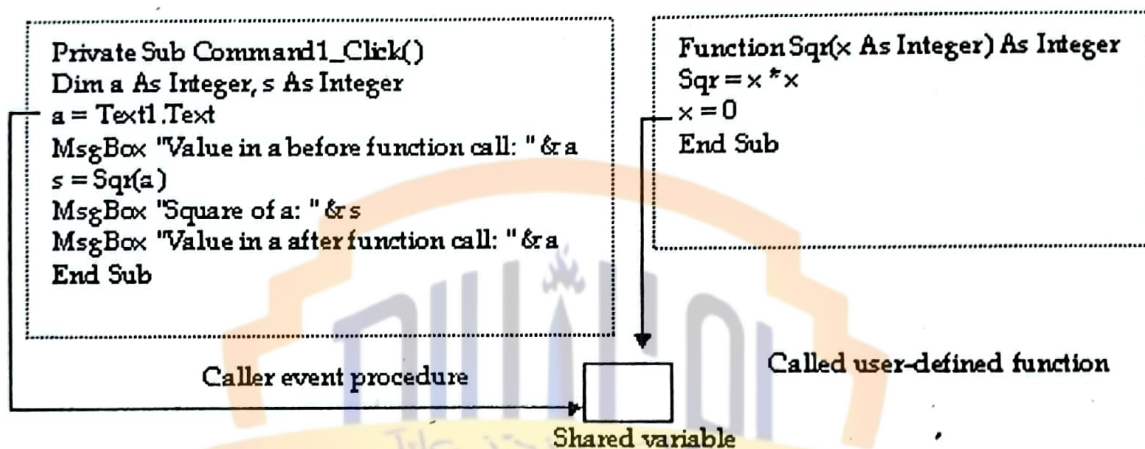


Figure: Shared Variable in Pass by Reference

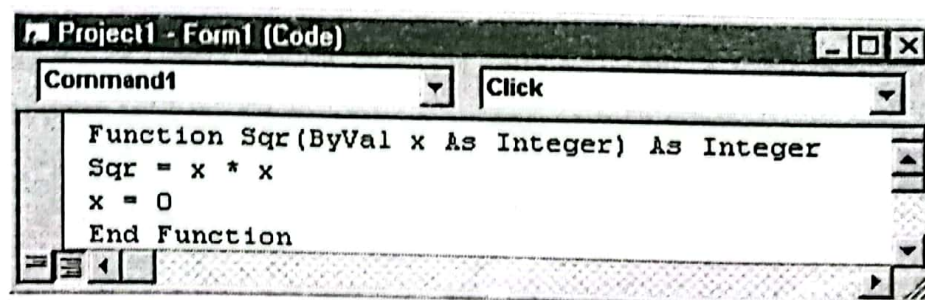
10.8.2 Pass by Value

When the arguments are passed by value, only the value of actual arguments is copied to the called procedure or function. If the procedure makes any change in the argument, it does not affect the actual argument. The keyword **ByVal** is used to indicate that the arguments are passed by value not by reference.

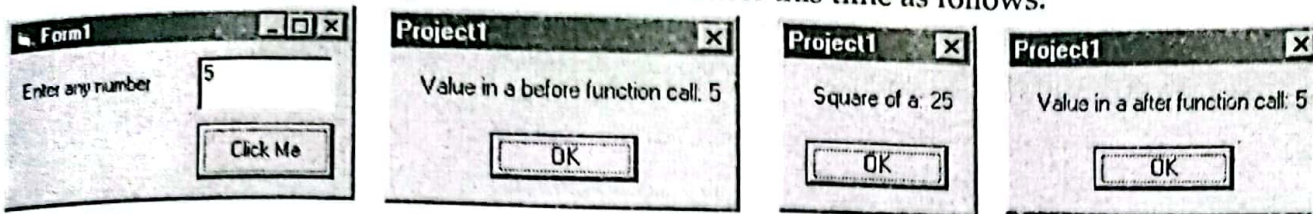
Program 10.9

Write a program that inputs a number in a textbox and passes it to a function by value. The function returns its square and sets its value to 0. The program displays the value of input before calling the function, its square and value after calling the function.

1. Use the same example used in the previous section. Add **ByVal** keyword in the function declaration as follows:



2. Run the project again. The result will differ this time as follows:



How It Works?

The above example passes the argument by value. It means that there are two different variables in the memory. The original value is stored in variable a. The value of a is passed to function that is stored in a separate variable x. A change in x does not change the value in a.

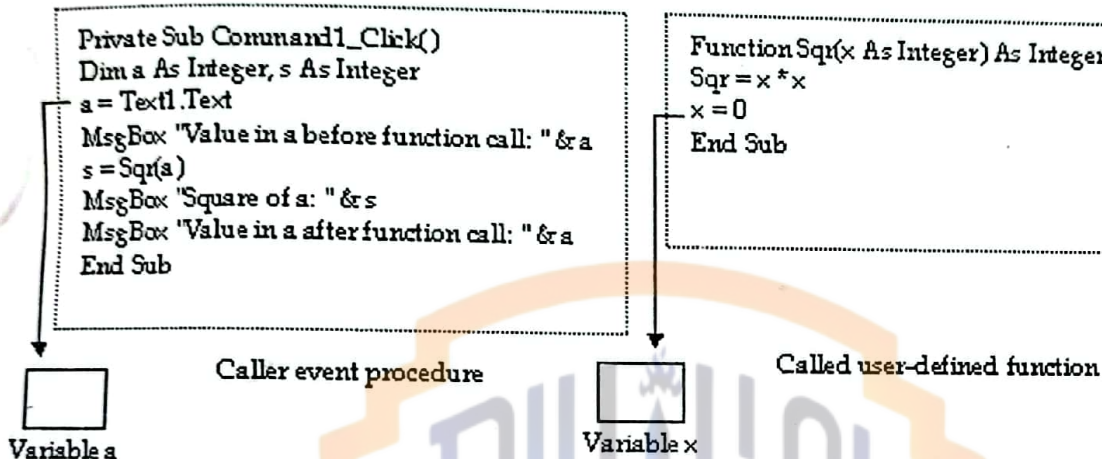


Figure: Separate Variables in Pass by Value

Difference between Sub Procedures and Function Procedures

The difference between Sub Procedure and Function Procedure is as follows:

Sub Procedure	Function Procedure
1. Sub procedure is declared with Sub keyword.	1. Function procedure is declared with Function keyword.
2. It cannot return a value.	2. It can return a value.
3. It called with Call keyword followed by the name of sub procedure.	3. It called only with the name of function procedure.

10.9 Optional Arguments

Visual Basic allows to create procedures or functions with optional arguments. If a procedure has optional arguments, it can be executed without providing the values for these arguments. The user has the option to provide or skip values for optional arguments. If an arguments is specified as optional, all subsequent arguments in the argument list must also be declared as optional. The keyword **Optional** is used to declare optional argument.

Syntax

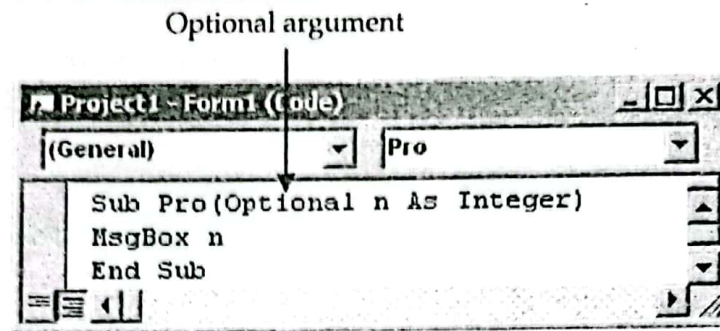
The syntax for declaring optional arguments is as follows:

```

Sub ProcedureName (Optional VariableName As DataType)
    Statement(s)
End Sub

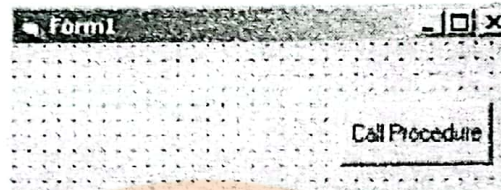
```


Example

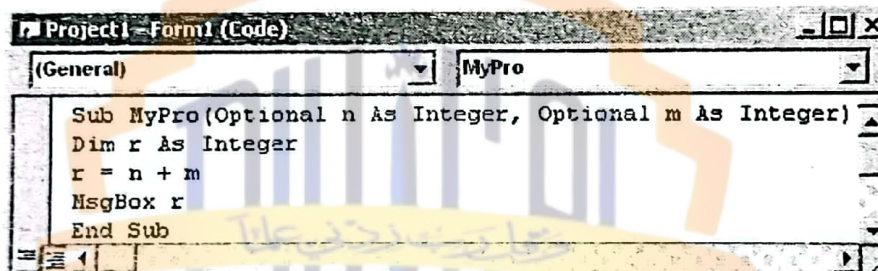
**Program 10.10**

Write a program to declare a subprogram that accept two optional parameters and displays their sum. The program calls it by providing different number of parameters.

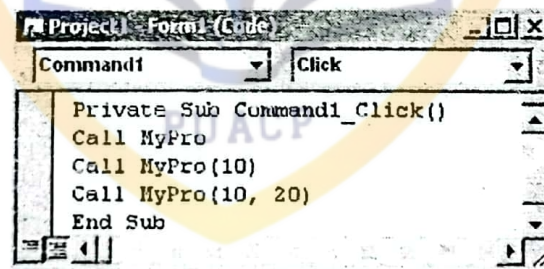
1. Start a new Standard EXE project.
2. Design the form as follows:



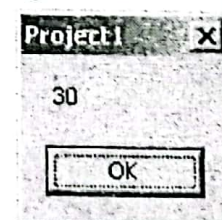
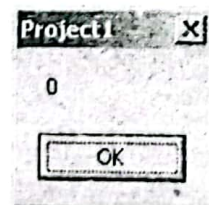
3. Declare the following sub procedure in **General Declaration** section:



4. Add the following code in Click event of the button:



5. Run the project and click the button. The following three messages will appear:

**How it Works?**

The above example declares a sub procedure **MyPro** with two optional arguments. When the user click the button, the procedure is called three times. The first call uses no argument so the result is 0. The second call passes value for one argument so the result is 10. The third call passes values for both arguments so the result is 30.

10.9.1 Default Values for Optional Arguments

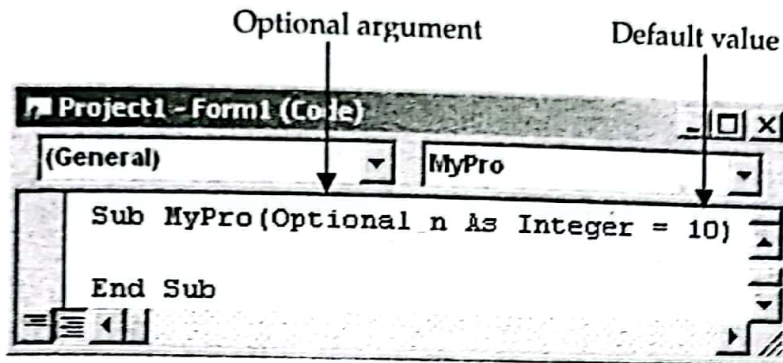
Visual Basic allows to specify the default values for optional arguments.

Syntax

The syntax of specifying default values for optional arguments is as follows:

```
Sub ProcedureName (Optional VariableName As DataType = Value)
Statement(s)
End Sub
```

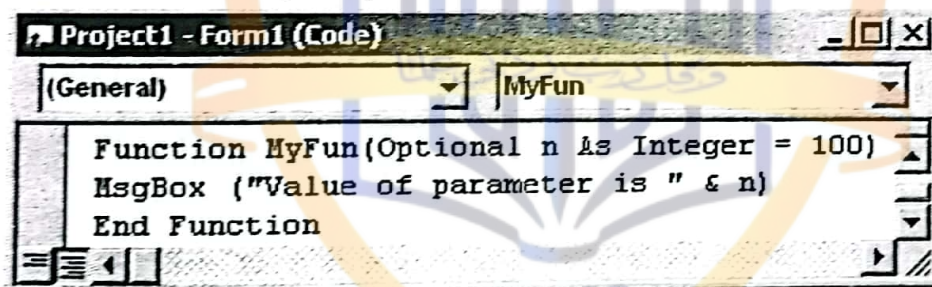
Example



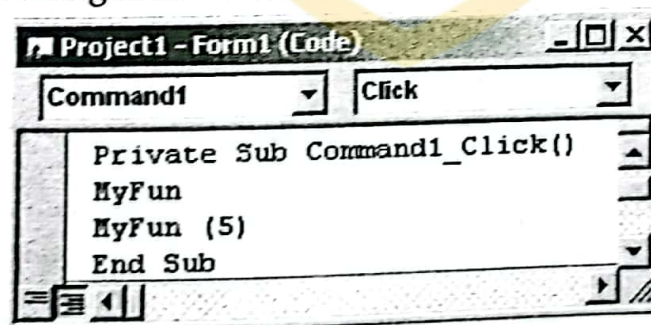
Program 10.11

Write a program to declare a function that accepts an optional parameter with a default value. The program calls the function two times.

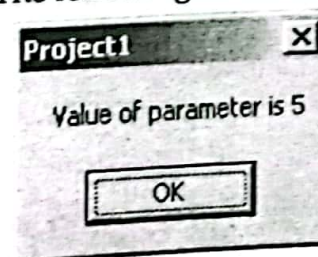
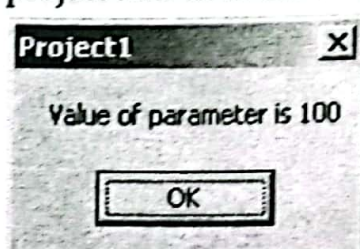
1. Start a new Standard EXE project.
2. Place a button on the form.
3. Declare the following sub procedure in General Declaration section:



4. Add the following code in Click event of the button:



5. Run the project and click the button. The following three messages will appear:



How it Works?

The above example declares a function **MyFun** with one optional arguments. When the user click the button, the function is called two times. The first call uses no argument so the default value 100 is displayed. The second call passes 5 so the value 5 is displayed.

10.10 Named Arguments

Visual Basic provides the option of **named arguments** as shortcut for typing argument values. With named arguments, the user can provide the arguments in any order by assigning a value to the named argument. It is performed by typing the argument name plus a colon followed by an equal sign and the value.

Syntax

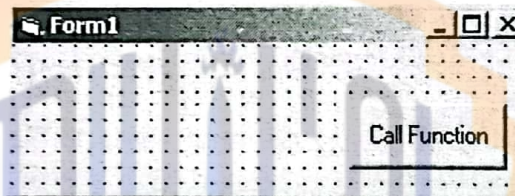
The syntax of using named arguments is as follows:

FunctionName(Argument:= Value)

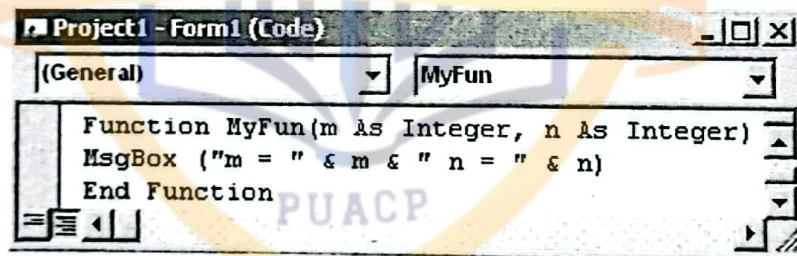
Program 10.12

Write a program to declare a function that accepts an two named parameters. The program calls the function two times by providing values in different sequence.

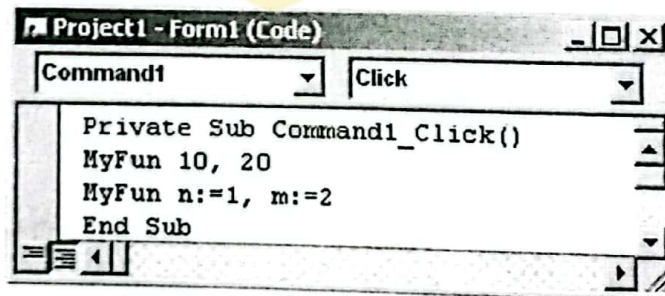
1. Start a new Standard EXE project.
2. Design the form as follows:



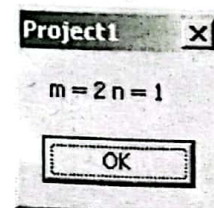
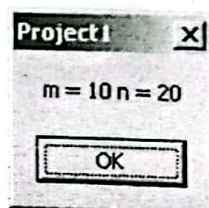
3. Declare the following sub procedure in General Declaration section:



4. Add the following code in Click event of the button:



5. Run the project and click the button. The following three messages will appear:



How it Works?

The above example declares a function **MyFun** with two arguments. When the user click the button, the function is called two times. The first call passes arguments without using names so 10 is stored in **m** and 20 in **n**. The second call passes 1 and 2 using named arguments so 1 is stored in **n** and 2 in **m**.

10.11 Indefinite Arguments

Generally, the number of arguments in the procedure call must be the same as in the procedure specification. The **ParamArray** keyword is used to specify a procedure that can accept any number of arguments.

Syntax

The syntax of using indefinite arguments is as follows:

```
FunctionName (ParamArray Argument( ))
    Statement(s)
End Function
```

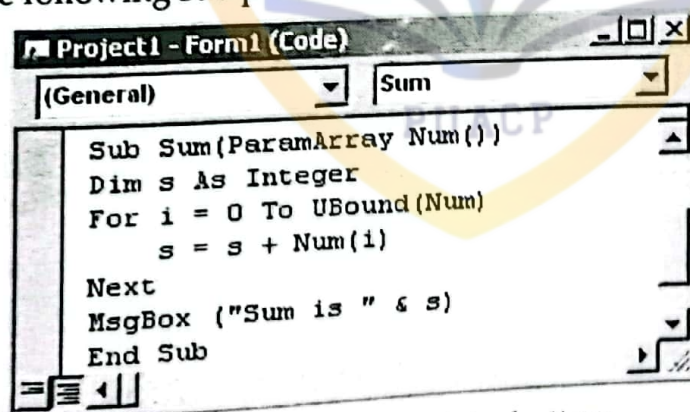
Program 10.13

Write a program to declare a subroutine that accepts any number of parameters and displays their sum. The program calls the subroutine by providing different values.

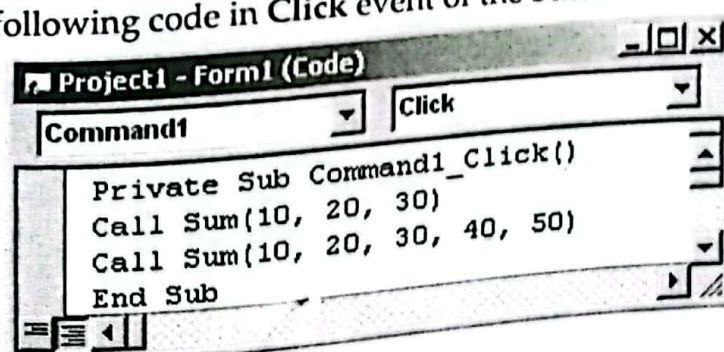
1. Start a new Standard EXE project.
2. Design the form as follows:



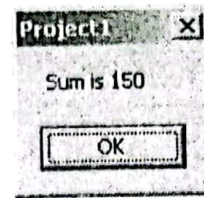
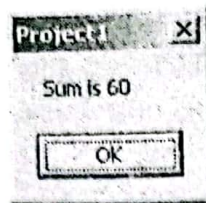
3. Declare the following sub procedure in General Declaration section:



4. Add the following code in Click event of the button:



5. Run the project and click the button. The following three messages will appear:



How it Works?

The above example declares a procedure **Sum** with indefinite arguments using **ParamArray** keyword. **Num** is the name of a variant array that is used to receive the arguments. When the user click the button, the function is called two times. The first call passes three arguments. The second call passes five arguments. The array **Num** receives these values and the loop calculates the some of these values and displays it.

10.12 Passing Array as Arguments

A procedure or function can accept an array as argument.

Syntax

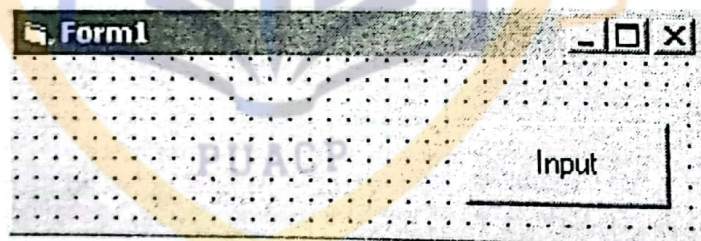
The syntax of using array as arguments is as follows:

```
FunctionName (Array( ) As DataType)
    Statement(s)
End Function
```

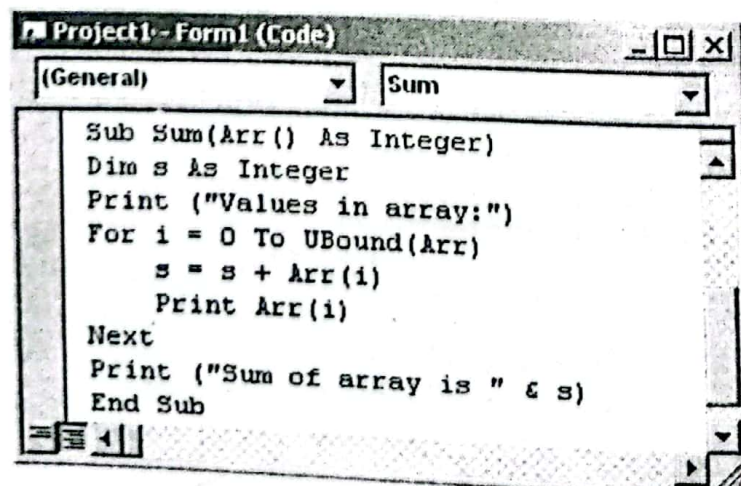
Program 10.14

Write a program that inputs five numbers in an array and passes the array to a sub procedure. The procedure displays all numbers in the array and their sum on the form.

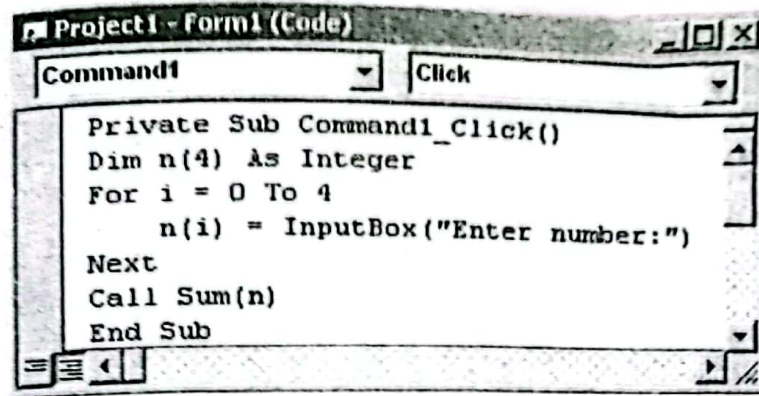
1. Start a new Standard EXE project.
2. Design the form as follows:



3. Declare the following sub procedure in General Declaration section:



4. Add the following code in Click event of the button:

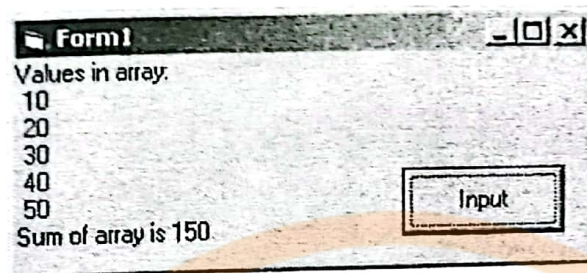


```

Private Sub Command1_Click()
    Dim n(4) As Integer
    For i = 0 To 4
        n(i) = InputBox("Enter number:")
    Next
    Call Sum(n)
End Sub

```

5. Run the project and click the button. The input box will appear five times.
6. Input five values. The result will appear as follows:



Form1

Values in array:

10
20
30
40
50

Sum of array is 150

Input

How it Works?

The above example declares a procedure **Sum** that accepts an array of integers. When the user click the button, the loop gets five inputs and stores in the array. The array is passed to the function. The function calculates the sum of the values and displays it on the form.

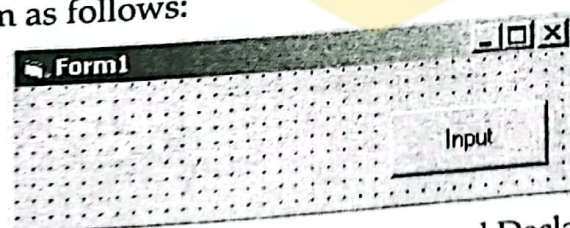
10.12.1 Passing Elements of Array as Arguments

Individual elements of an array can also be passed to procedure or function.

Program 10.15

Write a program that inputs five numbers in an array and passes these values to a function one by one. The function returns the square of value to print it.

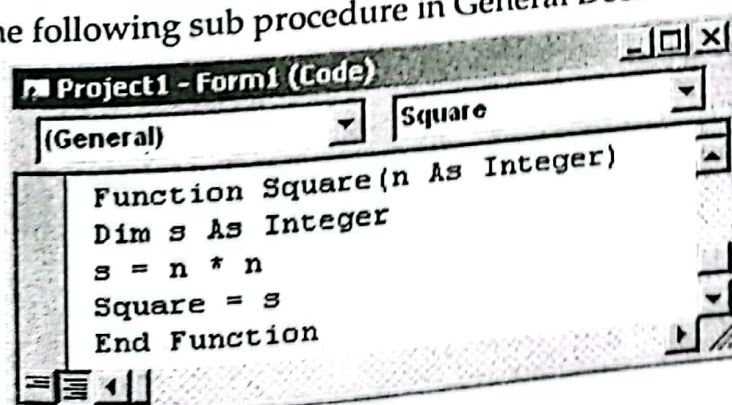
1. Start a new Standard EXE project.
2. Design the form as follows:



Form1

Input

3. Declare the following sub procedure in General Declaration section:

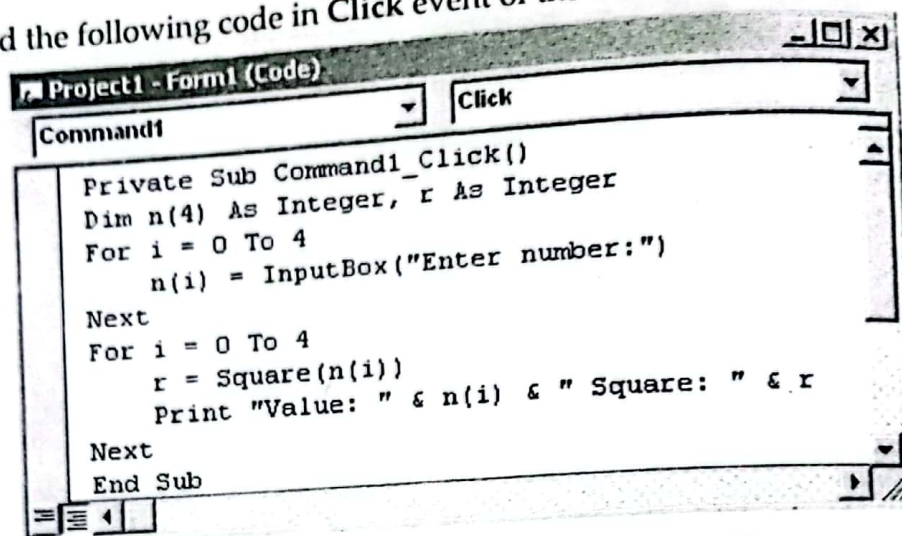


```

Function Square(n As Integer)
    Dim s As Integer
    s = n * n
    Square = s
End Function

```

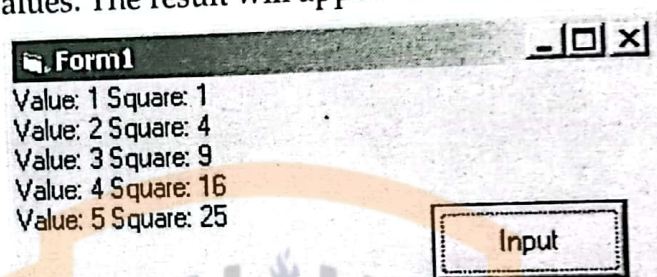

4. Add the following code in Click event of the button:



```

Project1 - Form1 (Code)
Click
Command1
Private Sub Command1_Click()
    Dim n(4) As Integer, r As Integer
    For i = 0 To 4
        n(i) = InputBox("Enter number:")
    Next
    For i = 0 To 4
        r = Square(n(i))
        Print "Value: " & n(i) & " Square: " & r
    Next
End Sub
  
```

5. Run the project and click the button. The input box will appear five times.
 6. Input five values. The result will appear as follows:



Form1

Value: 1 Square: 1
 Value: 2 Square: 4
 Value: 3 Square: 9
 Value: 4 Square: 16
 Value: 5 Square: 25

Input

How it Works?

The above example declares a procedure **Square** that accepts an integer. When the user click the button, the loop gets five inputs and stores in the array. The second loop calls the function five times. Each time an individual element of the array is passed to the function. The function calculates square of the value and returns it. The value of array and its square is displayed on the form.

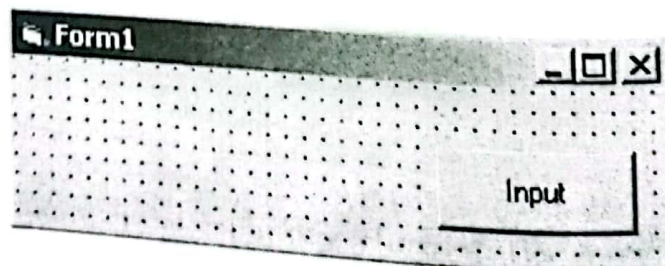
10.12.2 Returning Array from Function

A function or procedure can return an array.

Program 10.16

Write a program that inputs five numbers in array and passes the array to a function. The function calculates the square of each element in another array and returns the new array where the values are displayed on the form.

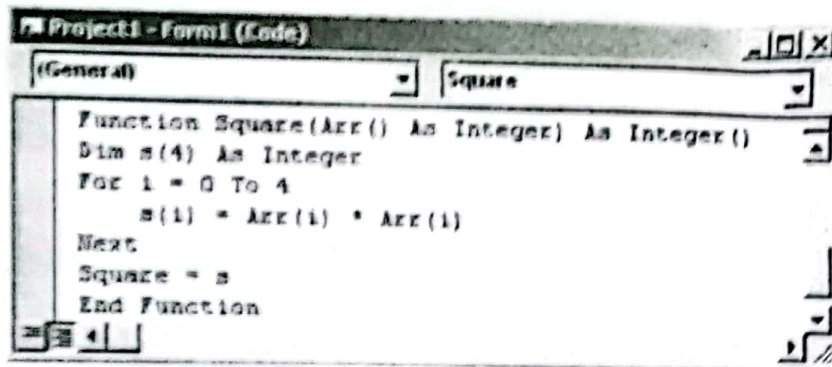
1. Start a new Standard EXE project.
2. Design the form as follows:



Form1

Input

3. Declare the following sub procedure in General Declaration section:

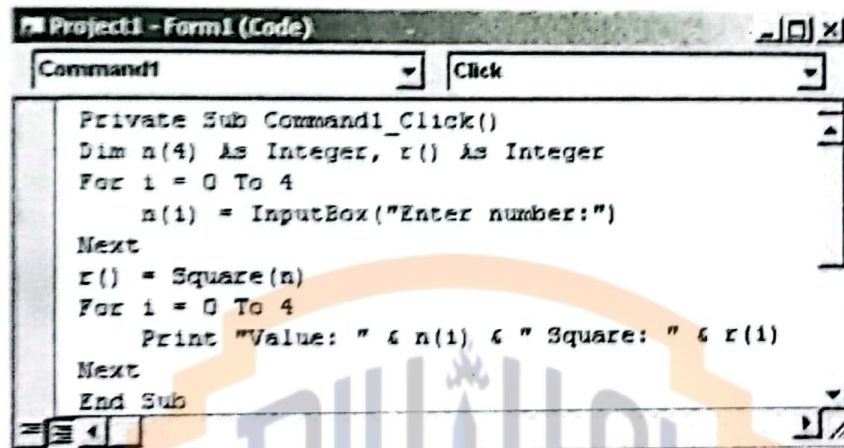


```

Function Square(Arr() As Integer) As Integer()
    Dim s(4) As Integer
    For i = 0 To 4
        s(i) = Arr(i) * Arr(i)
    Next
    Square = s
End Function

```

4. Add the following code in Click event of the button:



```

Private Sub Command1_Click()
    Dim n(4) As Integer, r() As Integer
    For i = 0 To 4
        n(i) = InputBox("Enter number:")
    Next
    r() = Square(n)
    For i = 0 To 4
        Print "Value: " & n(i) & " Square: " & r(i)
    Next
End Sub

```

5. Run the project and click the button. The input box will appear five times.
6. Input five values. The result will appear as follows:



```

Value: 1 Square: 1
Value: 2 Square: 4
Value: 3 Square: 9
Value: 4 Square: 16
Value: 5 Square: 25

```

How it Works?

The above example declares a procedure `Square` that accepts an integer array and returns an integer array. When the user click the button, the loop gets five inputs and stores in the array. The array is passed to the function. The function uses a loop and stores the squares of all elements in another array and returns that array. Finally, the values of both arrays are displayed on the form using loop.

10.13 Passing User-Defined Data Type as Argument

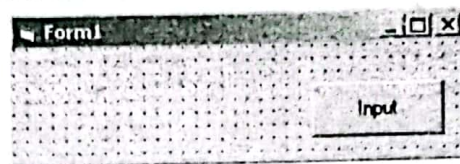
A user-defined data type can also be passed to procedures and functions as arguments.

Program 10.17

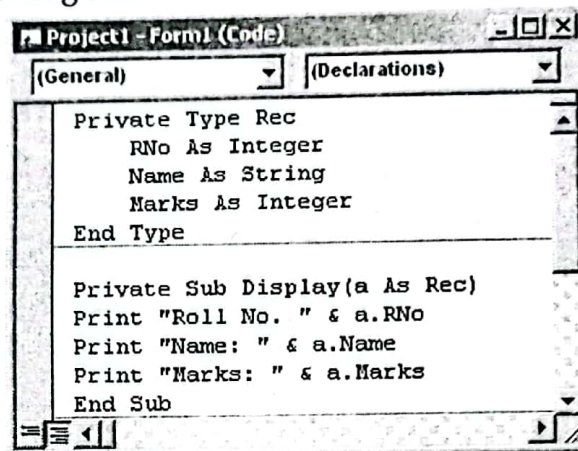
Write a program that declares a user-defined data type to store roll no, name and marks. It inputs values in the user-defined variable and passes it to a subroutine to display it.

1. Start a new Standard EXE project.

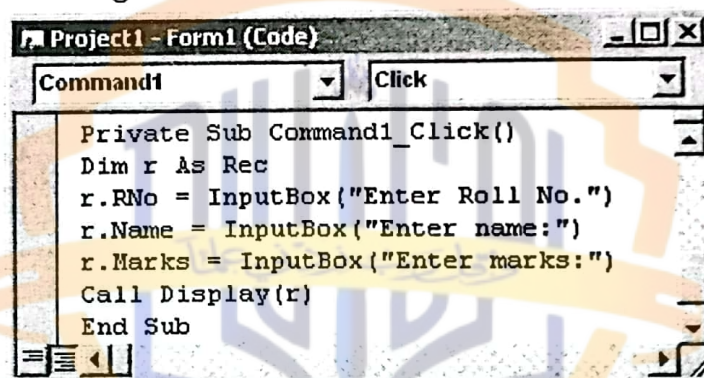
2. Design the form as follows:



3. Type the following code in General Declaration section:



4. Add the following code in Click event of the button:



5. Run the project and click the button. The input box will appear.
6. Input values. The result will appear as follows:

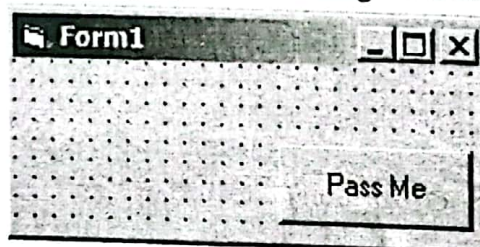


10.14 Passing Objects as Argument

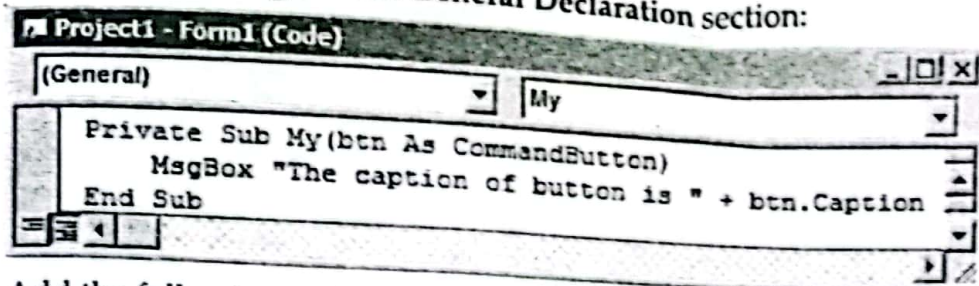
An object like button, textbox and form can also be passed to procedures and functions as arguments.

Program 10.18

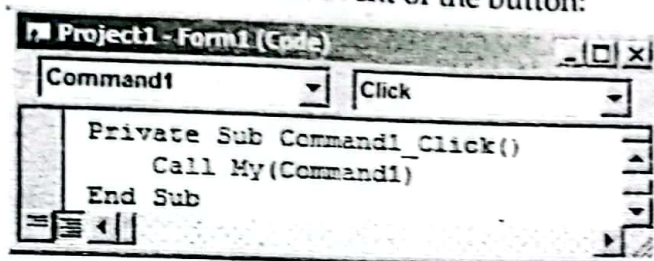
1. Start a new Standard EXE project and design the form as follows:



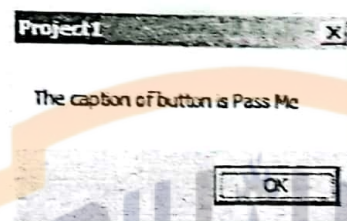
2. Type the following code in General Declaration section:



3. Add the following code in Click event of the button:



4. Run the project and click the button. The result will appear as follows:

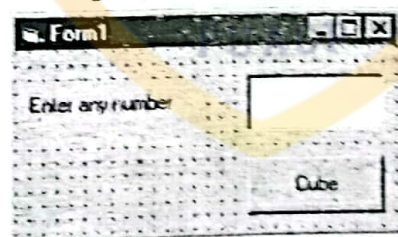


10.15 Exit Sub & Exit Function

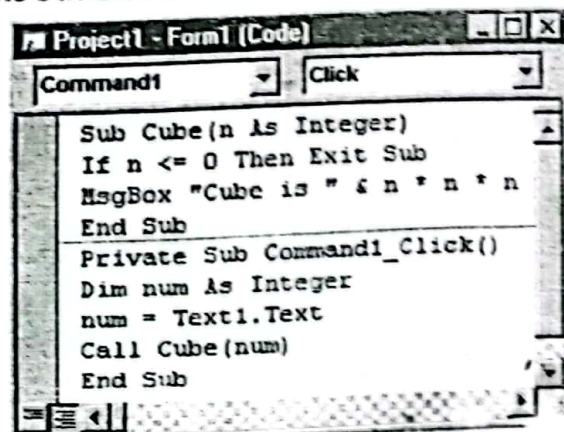
Exit Sub and Exit Function statements are similar to Exit For statement. Exit Sub is used inside a procedure to exit that procedure abnormally and Exit Function is used inside a function for the same purpose.

Program 10.19

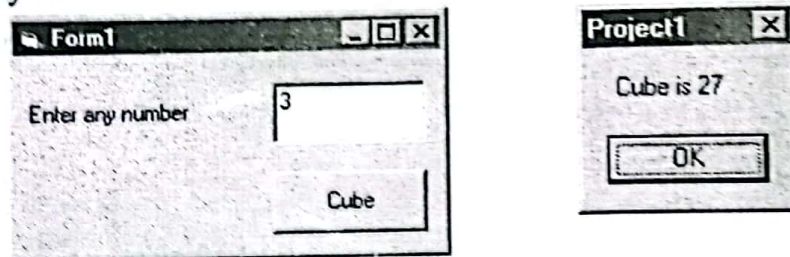
1. Start a new Standard EXE project and design the form as follows:



2. Double click the button and add the following code:



3. Run the project.
4. Enter any number in textbox and click button. The result will appear as follows:



How It Works?

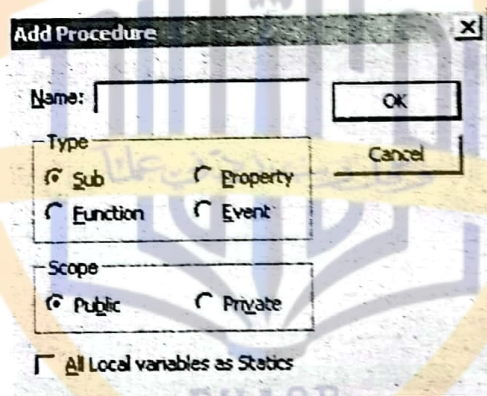
The above example uses a procedure that accepts a number as argument. If the argument is less than or equal to zero, the Exit Sub statement exits the control from the procedure and nothing happens. Otherwise, the cube of the number is displayed in the message box.

10.16 Adding Procedures using Menu

Procedure or function can be added in the project by using Tools menu. The following method can be used to add a procedure or function in Visual Basic project:

Example

1. Select Tools > Add Procedure. The following dialog box will appear:



2. Type any name for the procedure in Name box. Type "Test" for this example.
3. Select the type of the procedure. Use the default option of Sub.
4. Select the scope of the procedure. Use the default option of Public.
5. Select the checkbox at the bottom if to use all local variables as static variables.
6. Click OK. A blank procedure will be added in the form:

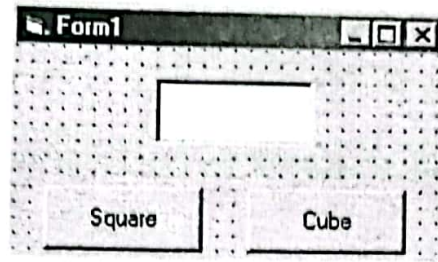
10.17 Modules

A module is a collection of variables, procedures and function that can be used from any part of the application. The basic purpose of creating a module is to access its code easily from anywhere. All variables, functions and procedures are declared with Public keyword for global access. Each module is stored independently in a file with .bas extension.

Program 10.20

Write a program that uses a module to add a function Compute. The function accepts an integer and computes its square or cube depending on the choice. The choice is set in a global variable when the user clicks a button.

1. Start a new Standard EXE project.
2. Design the form as follows:



3. Set the names of the buttons to `cmdSquare` and `cmdCube` respectively.
4. Select **Add Module** from **Project** menu. A module will be added in the project explorer.
5. Double click **Module1** in the **Project Explorer**. The code window will appear.
6. Add the following code:

```

Public Choice As String
Public Function Compute(n As Integer)
    If Choice = "Square" Then
        Compute = n * n
    ElseIf Choice = "Cube" Then
        Compute = n * n * n
    End If
End Function
  
```

7. Add the following code in Click event of `cmdSquare`:

```

Private Sub cmdSquare_Click()
    Module1.Choice = "Square"
    Dim num As Integer
    num = Text1.Text
    MsgBox "Square is " & Module1.Compute(num)
End Sub
  
```

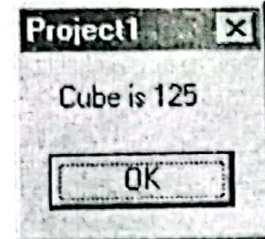
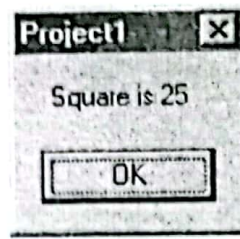
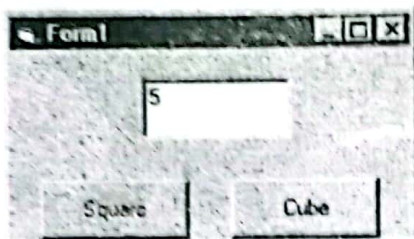
8. Add the following code in Click event of `cmdCube`:

```

Private Sub cmdCube_Click()
    Module1.Choice = "Cube"
    Dim num As Integer
    num = Text1.Text
    MsgBox "Cube is " & Module1.Compute(num)
End Sub
  
```

9. Run the project.

10. Add 5 in textbox and click both buttons. The results will appear as follows:



How It Works?

The above example uses a module for declaring a variable and a function. Both are declared with Public scope so that they can be used anywhere in the application. Both event procedures of the buttons use the global variable "Choice" for storing a string value to indicate the type of operation to apply on the data. The function in the module checks the value of this variable to check which button has called it.

10.17.1 Difference between Module and Procedure

A module is a component that can be added to a project. It provides a simple way to organize functions and other program elements such as variables etc. It is a set of variables, procedures and function that can be used from any part of the application. The basic purpose of creating a module is to access its code easily from anywhere. All variables, functions and procedures are declared with Public keyword for global access. Each module is stored independently in a file with .bas extension. A module contains code but no controls or other interface elements such as forms and controls.

A procedure is a set of statements that perform specific actions. Procedures improve the efficiency of programmer by breaking the program into smaller units that can be used in more than one place. A small, single task procedure makes code easier to reuse, manage, debug and enhance. There are two kinds of procedures. A Sub Procedure performs an action and a Function procedure performs an action and return a value.

Programming Exercise

1. Write a program that gets a number from user and displays the table of that number using procedure.
2. Write a program that gets a number from the user and displays its factorial using function.
3. Write a program that gets two numbers from the user and passes them to a procedure. The procedure displays all numbers between these numbers.
4. Write a program that gets two numbers from the user and calculates the result of first number raise to the power of second number using function.
5. Write a program to exchange the values of two textboxes using procedure.

Important Note: The solutions of all questions are available at the end of the book.

Short Questions

Q.1. Name two types of subprograms.

Two types of subprograms are sub procedures and function procedures.

Q.2. Write the general syntax for a procedure definition and a function definition.

The following general syntax defines a procedure:

```
Private Sub ProcedureName(Parameter As Type, ...)  
    statements
```

```
End Sub
```

The following general syntax defines a function:

```
Private Function FunctionName(Parameter As Type, ...) As ReturnType  
    statements
```

```
FunctionName = ReturnValue
```

```
End Function
```

Q.3. Write the general syntax for procedure call and function call.

The general syntax to call a procedure is as follows:

```
Call ProcedureName(argument1, argument2, ...)
```

A function is invoked by writing the function name along with the required arguments. The general syntax of function call where the returned value is assigned to a variable is as follows:

```
variableName = FunctionName(argument1, argument2, ...)
```

Q.4. Explain the General Declarations section of a form module?

The General Declarations section appears at the top of form's code module. The **Option Explicit** statement and any module level declarations may appear in the general declarations section.

Q.5. Consider the following codes:

```
Private Sub Command1_Click()
```

```
Dim a As Integer, b As Integer, c As Integer
```

```
a = 5
```

```
b = 6
```

```
c = (a * b) / 10
```

```
End Sub
```

```
Private Sub Command2_Click()
```

```
x = 40
```

```
y = x * c
```

```
Print y
```

```
End Sub
```

Q.6. What will be the output if 'Command1' and 'Command2' are clicked one after other?

The output will be 0. The variables declared within a procedure exist as long as the procedure is executing. When the procedure finishes, the value of the variable disappears. The value of a variable in a procedure is local to that procedure. It cannot be accessed another procedure.

Q.7. What is standard code module?

A standard code module holds sub procedures and function procedures to be used in multiple forms. It is also used to declare global variables and constants. It is a Visual Basic file with .BAS extension. Standard code modules contain no form but only code.

Q.8. Differentiate variable declaration in General Declarations section of standard code module and variable declaration in General Declarations section of form.

Global variables and constants must be declared in General Declarations section of a standard code module using Public statement. Variables declared with Dim statement in General Declarations section of a form code module are module level variables.

Q.9. Describe the rules for scope and lifetime of variables in a standard module.

Variables declared at the module level with Dim or Private statement are recognized by all procedures in the module but not in other modules. Variables declared with Public keyword are accessible to all open projects.

Q.10. What is the primary difference between sub procedure and function procedure?

A function procedure can return a value back to the calling procedure but sub procedure cannot return a value.

Q.11. What is a function argument?

An argument is a value that is passed to a function inside the function's parentheses.

Q.12. Explain the difference between ByVal and ByRef.

Arguments may be passed to a called procedure by value or by reference. When passed ByVal, only a copy of original value is passed. The called procedure cannot alter the original value. When data is passed ByRef, the memory address of original value is passed to the procedure. It allows the procedure to change the original value.

Q.13. What is the default parameter passing mechanism in Visual Basic?

The default mechanism for passing parameters to procedures and functions in Visual Basic is by reference.

Q.14. What is modular programming? List some of its advantages.

Modular programming is programming in a component-based design. It uses subprograms. This technique provides a logical organization of large programs. It enhances the readability of code and allows easier debugging and promotes code reuse.

Q.15. What are different types of procedure?

There are three types of procedures called event procedure, Sub procedure and Function procedure. An event procedure performs a set of statements. It is executed when the appropriate event is triggered. A Sub procedure also performs a set of statements. It is executed when the code in another procedure call the procedure by name. A Function procedure is similar to a Sub except that it also returns a value. Sub and function procedures are collectively recognized as general procedures.

Q.16. What is the difference between even procedure and user-defined sub procedure?

An event procedure is a procedure that belongs to a specific object and event. A user-defined sub procedure is a procedure that is not associated with a specific object. It can be called by more than one object or procedure.

Q.17. What is the use of function procedure?

A function procedure performs a set of tasks and returns a value. It is a set of statements enclosed by Function and End Function statements. It is very similar to a user-defined sub procedure except that a function procedure returns a value to the procedure that calls it. It is used on the right side of assignment operator. It can also accept arguments passed to it by the calling procedure. The datatype specifies type of data that the function will return. A function is invoked by using its name.

Q.18. Differentiate between modules and procedures?

A module is a set of variables, procedures and function that can be used from any part of the application. It is used to access code easily from anywhere. A module contains code but no controls. A procedure is a set of statements that perform specific actions. Procedures improve the efficiency of programmer by breaking the program into smaller units that can be used in more than one place.

Multiple Choice

1. Sub and Function procedures are triggered by:
 - a. An event such as clicking a button.
 - b. Referring to the name of the procedure in code
 - c. Both a and b
 - d. None
2. The key difference between a Sub procedure and a Function is:
 - a. Sub can return value but Function cannot
 - b. Function can returns value but Sub cannot
 - c. Both a and b
 - d. None
3. The value returned by a function can be:
 - a. Used for processing
 - b. Assigned to a property
 - c. Both a and b
 - d. Ignored
4. An actual argument is:
 - a. The argument inside the procedure
 - b. an argument at the time the procedure is called
 - c. Both a and b
 - d. None
5. Executing an 'Exit Sub' statement in a procedure will:
 - a. Stop further execution within that routine
 - b. Stop all code execution and exit program
 - c. Force all variables to be re-initialized
 - d. Generate a 'Run-time error'
6. Which event is good to initialize form-scope variables?
 - a. Form_GotFocus
 - b. Form_Change
 - c. Form_Activate
 - d. Form_Load
7. A variable's lifetime refers to:
 - a. How long it remains in memory
 - b. Its data type
 - c. How widely it is recognized to procedures in project
 - d. Whether it is reinitialized every time a procedure is called
8. Variables in a standard module keep their values:
 - a. Until the form that created module is closed
 - b. For the life of the project
 - c. Until the user shuts down the computer
 - d. During procedure which refers to them
9. The code that is used in a sub procedure to stop execution of a program is:
 - a. Exit
 - b. Halt
 - c. End
 - d. End Sub
10. Which keyword is used to trigger the execution of another procedure?
 - a. Go
 - b. Call
 - c. Start
 - d. Run
11. If you want a procedure to be able to change the values of parameters passed into it, use:
 - a. ByVal keyword
 - b. ByRef keyword
 - c. Address keyword
 - d. All
12. How widely a variable is recognized to procedures in the project refers to its:
 - a. Scope.
 - b. Lifetime.
 - c. Duration.
 - d. Execution
13. Which keyword is used to make a variable viable to all modules in the project?
 - a. Public
 - b. Private
 - c. Dim
 - d. None
14. What three keywords are used to declare local, form-scope and global variables?
 - a. Private, private, private
 - b. Local, Module, Global
 - c. Private, Public, Dim
 - d. Dim, Private, Public
15. What is false about the form level variable?
 - a. New value can be re-assigned to it anywhere in the program
 - b. It exists for all event procedures and user-defined procedures
 - c. It is declared outside event procedures or user-defined procedures
 - d. No local variable could have the same name
16. You can force a variable to preserve its value between procedure calls by using ___ keyword.
 - a. Default
 - b. Static
 - c. Volatile
 - d. None

17. A value can be returned by the function procedure using:
 a. Function Name=Value b. Return value
 c. Cannot return a value d. None
18. A user defined subprogram returns
 a. No value b. Exact one value
 c. Up to three values d. As many values as the programmer likes.
19. _____ is a value passed to a function:
 a. Argument b. Function c. Message box d. None
20. A variable declared in a code module's general declaration section is called a:
 a. Local variable b. Global variable
 c. Form level variable d. Remote variable

Answers

1. b	2. b	3. c	4. b	5. a	6. d
7. a	8. b	9. c	10. b	11. b	12. a
13. a	14. d	15. d	16. b	17. a	18. a
19. a	20. b				

Fill in the Blanks

- If a program consists of only one form, _____ variables persist for the lifetime of the program but cannot be accessed by other procedures in the same form.
- A procedure or function that is part of a control is called a _____.
- The keyword _____ is used to declare global scope procedures and functions.
- A variable passed to a procedure is called an _____.
- _____ and _____ are the types of procedures.
- The code statement used to exit a function before normal "End Function" statement is _____.
- To make a function accessible to all modules, use the _____ modifier.

Answers

1. Static	2. Method	3. Public
4. Parameters	5. Sub, Function	6. Exit Sub
7. Public	8.	9.

True / False

- Parameters are optional in a sub procedure
- A function procedure is identical to a sub procedure
- To pass control out of a procedure before End Sub, use the Exit Sub statement.
- In structured programming, there should be many ways to enter and exit a structure.

17. A value can be returned by the function procedure using:
 a. Function Name=Value b. Return value
 c. Cannot return a value d. None
18. A user defined subprogram returns _____.
 a. No value b. Exact one value
 c. Up to three values d. As many values as the programmer likes.
19. _____ is a value passed to a function:
 a. Argument b. Function c. Message box d. None
20. A variable declared in a code module's general declaration section is called a:
 a. Local variable b. Global variable
 c. Form level variable d. Remote variable

Answers

1. b	2. b	3. c	4. b	5. a	6. d
7. a	8. b	9. c	10. b	11. b	12. a
13. a	14. d	15. d	16. b	17. a	18. a
19. a	20. b				

Fill in the Blanks

- If a program consists of only one form, _____ variables persist for the lifetime of the program but cannot be accessed by other procedures in the same form.
- A procedure or function that is part of a control is called a _____.
- The keyword _____ is used to declare global scope procedures and functions.
- A variable passed to a procedure is called an _____.
- _____ and _____ are the types of procedures.
- The code statement used to exit a function before normal "End Function" statement is _____.
- To make a function accessible to all modules, use the _____ modifier.

Answers

1. Static	2. Method	3. Public
4. Parameters	5. Sub, Function	6. Exit Sub
7. Public	8.	9.

True / False

- Parameters are optional in a sub procedure.
- A function procedure is identical to a sub procedure.
- To pass control out of a procedure before End Sub, use the Exit Sub statement.
- In structured programming, there should be many ways to enter and exit a structure.

5. Modularization is when a larger problem is divided into smaller sections that are easier to understand and maintain.
6. Static variables keep their value until another procedure is called in the program.
7. Module-level variables have wider scope than global variables.
8. Global variables should be used as much as possible.
9. It is good coding practice to initialize variables at the beginning of a procedure.
10. The area in a program in which a variable is available for use is called the **scope** of variable.
11. Passing arguments by value means the called procedure receives a copy of the calling procedure's argument.
12. By default, Visual Basic passes arguments by value.
13. Module variables can be accessed from any procedure in a module.
14. Whenever possible, use global variables than arguments to communicate with procedures.
15. Functions can be invoked using Call statement when function's return value is to be discarded.
16. There is no way for a procedure to return a value to a calling program.
17. Passing an argument by value gives a procedure only a copy of the variable.
18. The scope of a variable defines which part of the code is aware of its existence.
19. Sub procedures and Function procedures share the same characteristics, with one important difference that Sub procedure can return a value.
20. Procedure-level variables are also known as **local variables**.
21. Sub and Function procedures are referred to as general procedures.
22. Module is a type of sub program.
23. The items in parentheses of a Sub procedure are called assignments.
24. When information is passed from one Sub procedure to another and then passed back to the Call statement, it is called pass by reference.
25. Any variable that is not declared in a function is said to be form level.
26. When an argument is passed by value, a copy of the argument's value is passed. The called procedure can manipulate that copy but cannot manipulate the caller's data.
27. With call-by-reference, the caller gives the called procedure the ability to directly access the caller's data and to modify that data if the called procedure so chooses.

Answers

1. T	2. F	3. T	4. F
5. T	6. F	7. F	8. F
9. T	10. T	11. T	12. F
13. T	14. F	15. T	16. F
17. T	18. T	19. F	20. T
21. T	22. T	23. F	24. T
25. F	26. T	27. T	

Project 6

Change Password

Project Specification

The project Change Password uses the following forms:

- **Password Form:** It appears when the project is started. It inputs password from the user
- **Main Form:** It appears if the user enters valid password. It contains different buttons.
- **Change Password Form:** It appears when the user click Change Password button. It is used to change the password of the user.

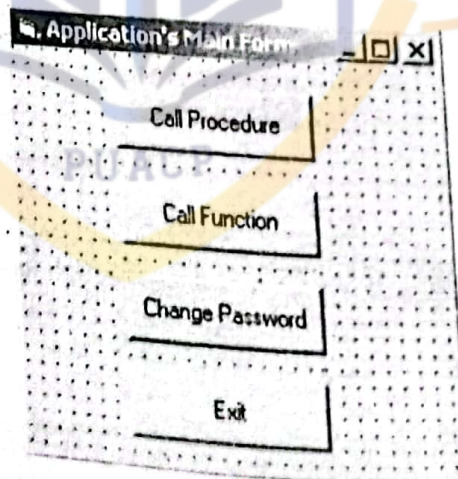
The project also uses a module that contains a variable and three procedures.

Procedure

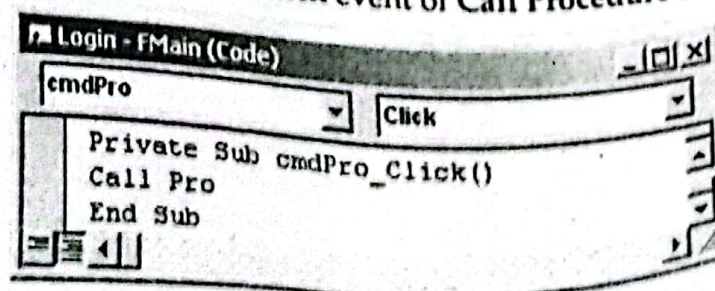
1. Start a new Standard EXE project.
2. Set the properties of Form1 as follows:

Control	Control Name	Property	Value
Form	FMain	Caption	Application's Main Form
CommandButton	cmdPro	Caption	Call Procedure
CommandButton	cmdFun	Caption	Call Function
CommandButton	cmdChange	Caption	Change Password
CommandButton	cmdExit	Caption	Exit

3. Design the form as follows:



4. Add the following code in Click event of Call Procedure button:



5. Add the following code in Click event of Call Function button:

```

Login - FMain (Code)
cmdFun Click
Private Sub cmdFun_Click()
Fun
End Sub
    
```

6. Add the following code in Click event of Change Password button:

```

Login - FMain (Code)
cmdChange Click
Private Sub cmdChange_Click()
FChangePassword.Show
End Sub
    
```

7. Add the following code in Click event of Exit button:

```

Login - FMain (Code)
cmdExit Click
Private Sub cmdExit_Click()
Call ExitProject
End Sub
    
```

8. Add another form in the project and set its properties as follows:

Control	Control Name	Property	Value
Form	Flogin	Caption	Login
CommandButton	cmdLogin	Caption	Login
CommandButton	cmdCancel	Caption	Cancel
Label	lblPassword	Caption	Password
Textbox	txtPassword	Text	Blank

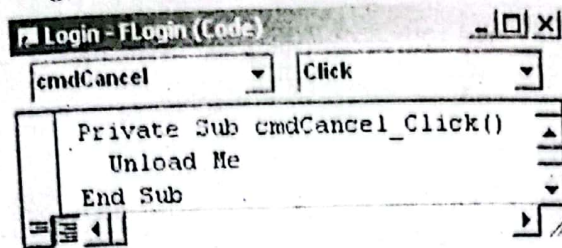
9. Design the form as follows:

10. Add the following code in Click event of Login button:

```

Login - FLogin (Code)
cmdLogin Click
Private Sub cmdLogin_Click()
If txtPassword.Text = Pass Then
Unload Me
FMain.Show
Else
MsgBox "Wrong password. Try again."
txtPassword.SetFocus
txtPassword.SelStart = 0
txtPassword.SelLength = Len(txtPassword.Text)
End If
End Sub
    
```

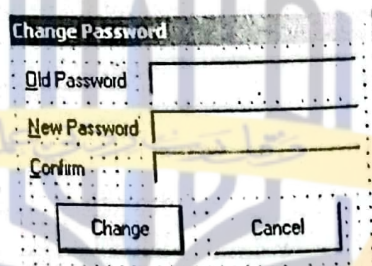

11. Add the following code in Click event of Cancel button:



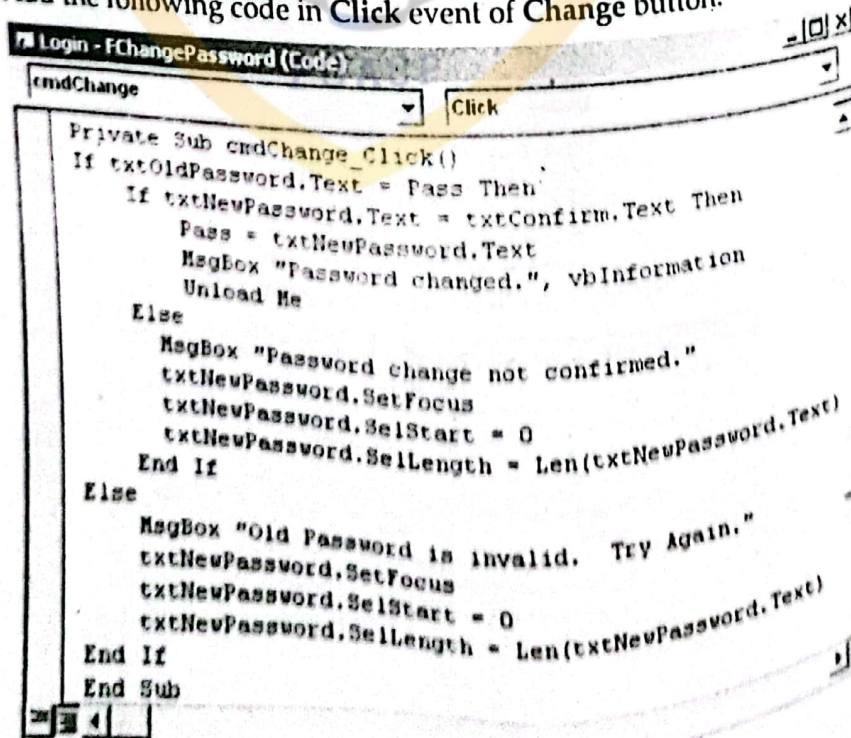
12. Add another form in the project and set its properties as follows:

Control	Control Name	Property	Value
Form	FchangePassword	Caption	Change Password
Label	lblOldPassword	Caption	Old Password
Label	lblNewPassword	Caption	New Password
Label	lblConfirm	Caption	Confirm
Textbox	txtOldPassword	Text	Blank
Textbox	txtNewPassword	Text	Blank
Textbox	txtConfirm	Text	Blank
CommandButton	cmdChange	Caption	Change
CommandButton	cmdCancel	Caption	Cancel

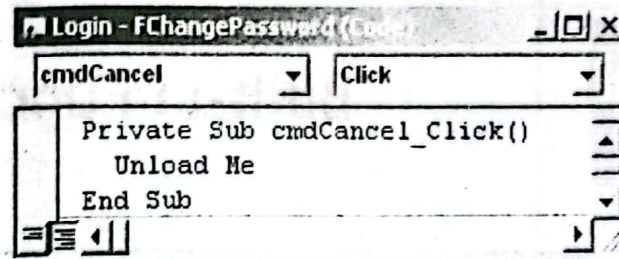
13. Design the form as follows:



14. Add the following code in Click event of Change button:

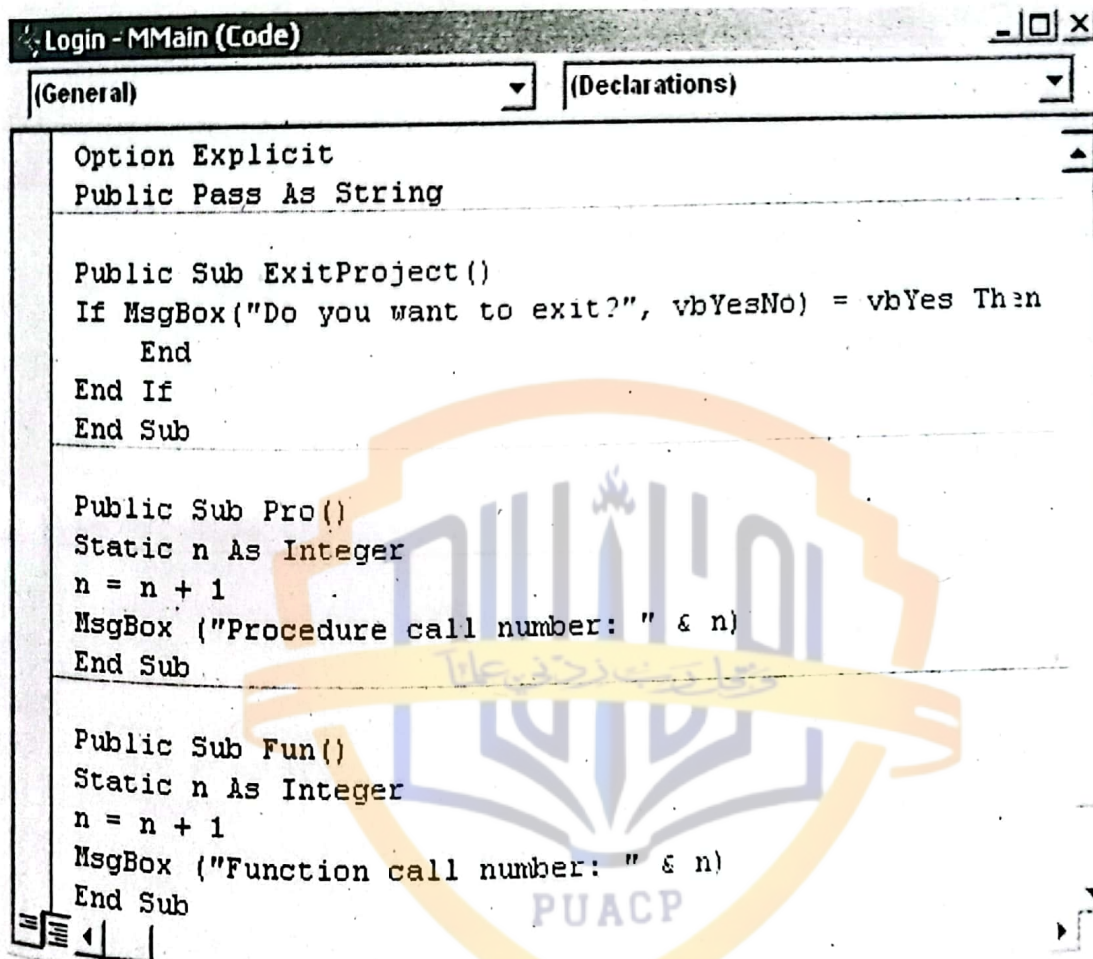


15. Add the following code in Click event of Cancel button:



```
Private Sub cmdCancel_Click()  
    Unload Me  
End Sub
```

16. Add a module MMain and add the following code:



```
Option Explicit  
Public Pass As String  
  
Public Sub ExitProject()  
    If MsgBox("Do you want to exit?", vbYesNo) = vbYes Then  
        End  
    End If  
End Sub  
  
Public Sub Pro()  
    Static n As Integer  
    n = n + 1  
    MsgBox ("Procedure call number: " & n)  
End Sub  
  
Public Sub Fun()  
    Static n As Integer  
    n = n + 1  
    MsgBox ("Function call number: " & n)  
End Sub
```